Replay-Reduced Continual Reinforcement Learning with Critic-Based Retention

1 Introduction

Continual reinforcement learning (CRL) trains an agent on a sequence of tasks. Its central difficulty is catastrophic forgetting: adapting a policy to the current task can degrade performance on tasks learned previously. Replay addresses this problem by retaining past trajectories or repeatedly collecting new data, but replay-buffer storage grows with the task sequence and retaining a task-specific actor for every task can also be expensive.

We consider a replay-reduced setting in which each previously encountered task simulator remains available, but no past trajectories, replay buffers, or task-specific actors are retained. After learning task i, the learner stores only a frozen critic $\widehat{V}_i$ that approximates the value of the task-specific policy learned on that task. When a later task arrives, the learner can reset an old simulator to generate fresh anchor states and use $\widehat{V}_i$ as a fixed reference. Thus, the method replaces stored experience with short, on-demand simulator rollouts and a compact task-specific value reference. This does *not* remove all old-task interaction: it reduces retained data, rather than providing a simulator-free guarantee.

At each stage, the method separates adaptation from consolidation. First, the incoming global policy is adapted to the new task, producing a local policy. Then a candidate global policy is initialized from the incoming global policy and optimized jointly across tasks. Consolidation has three goals: retain the local policy's performance on the current task, avoid falling below the stored critic reference at selected states of every old task, and, when full old-task evaluations are affordable, improve the aggregate old-task return relative to the incoming global policy. The first two goals are enforced as constraints; the third is an objective.

The critic-based condition is deliberately local. It evaluates a candidate policy over a short rollout from each anchor and compares the bootstrapped return with the frozen critic at the same state. Consequently, it is an empirical retention surrogate, not a universal no-forgetting guarantee. Its quality depends on critic accuracy and on the coverage of the generated anchor states. The formulation below makes these assumptions explicit, distinguishes the fixed reference quantities from trainable ones, and gives a direct primal-dual implementation.

2 Methodology

2.1 Sequential learning setting

At stage k, the learner has encountered tasks $1, \dots, k$. Task i is a discounted MDP

$$M_i = (\mathcal{S}_i, \mathcal{A}, P_i, r_i, \rho_i, \gamma),$$

where the tasks share a policy interface (the same action space and compatible observations), ρ_i is the initial-state distribution, and $\gamma \in (0, 1)$. For a stationary policy π, define its state value and initial-state value on task i as

$$V_i^\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_i(s_t, a_t) \mid s_0 = s, \pi\right], \qquad V_i^\pi = \mathbb{E}_{s_0 \sim \rho_i}[V_i^\pi(s_0)]. \tag{1}$$

Let $\pi_G^- = \pi_{\phi^-}$ be the global policy entering stage k. Training it on M_k alone yields the local policy $\pi_L = \pi_{\theta^\star}$. Consolidation optimizes $\pi_\phi = \pi_\phi$, initialized at $\phi^{(0)} = \phi^-$, and returns $\pi_G^+ = \pi_{\phi^\star}$. Once task i has been learned, its local actor $\pi_L^{(i)}$ is discarded and its frozen critic is retained:

$$\widehat{V}_i(s) \approx V_i^{\pi_L^{(i)}}(s). \tag{2}$$

The current local policy is needed only during its stage-k consolidation and is discarded afterward. Previous tasks contribute their simulators and critics, but not stored anchor states: anchors are regenerated for each consolidation stage and discarded when that stage ends.

2.2 Why the consolidation objective focuses on old tasks

For nonnegative weights ω_i with $\sum_i \omega_i = 1$, define the aggregate score

$$J_k(\pi) = \sum_{i=1}^{k} \omega_i V_i^{\pi}. \tag{3}$$

Adding and subtracting the local-policy score gives

$$\begin{aligned} J_k(\pi_G^+) - J_k(\pi_G^-) = \omega_k\big(V_k^{\pi_L} - V_k^{\pi_G^-}\big) + \omega_k\big(V_k^{\pi_G^+} - V_k^{\pi_L}\big) \\ + \sum_{i=1}^{k-1} \omega_i\big(V_i^{\pi_G^+} - V_i^{\pi_G^-}\big). \end{aligned} \tag{4}$$

The first term measures the gain obtained by local adaptation and is constant during consolidation. The second term measures whether consolidation loses current-task performance. We control it with a constraint. This leaves the improvement on earlier tasks as the consolidation objective:

$$R_k(\phi) = \sum_{i=1}^{k-1} \omega_i\big(V_i^{\pi_\phi} - V_i^{\pi_G^-}\big). \tag{5}$$

Estimating (5) requires ordinary full-episode (or otherwise unbiased) old-task return estimates. Short rollouts are used only for the retention constraints below; they are not a substitute for the full-return objective.

2.3 Current-task protection

We allow a small loss relative to the local policy but do not penalize improvements beyond it. The one-sided squared violation is

$$F_k(\pi_\phi) = \big[V_k^{\pi_L} - V_k^{\pi_\phi}\big]_+^2 \le \epsilon, \tag{6}$$

where $\epsilon \ge 0$ controls the allowed squared value gap. In particular, $\epsilon = 0$ recovers the direct condition $V_k^{\pi_\phi} \ge V_k^{\pi_L}$. The hinge form is useful because it has zero gradient whenever the candidate already matches or exceeds the local reference.

2.4 Critic-based retention on earlier tasks

Before consolidating stage k, we generate a finite anchor set $X_i \subset \mathcal{S}_i$ for each $i < k$. To obtain anchors at different episode depths, we reset M_i, roll out the *fixed* incoming policy π_G^- for a random number of steps (for example, geometrically distributed and clipped at termination), and retain the reached state. Sampling under a fixed policy and holding X_i fixed during one consolidation phase prevents the constraint set from changing with ϕ.

For an anchor $s \in X_i$, roll out π_ϕ for n steps in M_i and define the bootstrapped residual

$$\Delta_{i,n}(s;\phi) = \mathbb{E}\left[\sum_{j=0}^{n-1} \gamma^j r_i(s_j, a_j) + \gamma^n \widehat{V}_i(s_n) \,\middle|\, s_0 = s, \pi_\phi\right] - \widehat{V}_i(s). \tag{7}$$

If $\widehat{V}_i$ were exact for the candidate policy, this quantity would be zero. Here it is a comparison against the stored local-policy critic: requiring it not to be too negative asks the candidate's n-step bootstrapped return to remain close to, or above, the stored local reference at the anchor.

The residual is also the expected sum of one-step temporal-difference terms. With

$$\delta_t = r_i(s_t, a_t) + \gamma \widehat{V}_i(s_{t+1}) - \widehat{V}_i(s_t), \tag{8}$$

the terms telescope:

$$\sum_{j=0}^{n-1} \gamma^j \delta_j = \sum_{j=0}^{n-1} \gamma^j r_i(s_j, a_j) + \gamma^n \widehat{V}_i(s_n) - \widehat{V}_i(s_0). \tag{9}$$

For one tolerance $\tau_i \geq 0$ per old task, enforce the worst-anchor condition

$$C_i(\phi) \coloneqq \max_{s \in X_i} \{\tau_i - \Delta_{i,n}(s;\phi)\} \leq 0, \qquad i = 1, \ldots, k-1. \tag{10}$$

Equivalently, $\Delta_{i,n}(s;\phi) \geq -\tau_i$ for every anchor. Smaller τ_i produces stricter retention; choosing $\tau_i = 0$ requires each estimated bootstrapped return to meet the critic reference.

The resulting stage-k program is

$$\begin{aligned} \max_{\phi} \quad & R_k(\phi) \\ \text{s.t.} \quad & F_k(\pi_\phi) \leq \epsilon, \\ & C_i(\phi) \leq 0, \qquad i = 1, \ldots, k-1. \end{aligned} \tag{11}$$

2.5 Primal-dual optimization

The constrained problem (11) has two kinds of variables. The *primal* variable is ϕ: changing it changes the candidate policy and therefore the return and constraint values. The *dual* variables are nonnegative multipliers: μ is the price assigned to a current-task violation, and λ_i is the price assigned to a violation on old task i. A multiplier becomes large only when its associated constraint is repeatedly violated, causing the next policy update to place more weight on satisfying that constraint. This construction gives the Lagrangian

$$\mathcal{L}(\phi, \mu, \lambda) = R_k(\phi) - \mu\big(F_k(\pi_\phi) - \epsilon\big) - \sum_{i=1}^{k-1} \lambda_i C_i(\phi), \qquad \max_{\phi} \min_{\mu, \lambda \succeq 0} \mathcal{L}. \tag{12}$$

We ascend in the primal variable, which improves the Lagrangian, and descend in the dual variables, which increases the penalty on violated constraints. The remaining question is how to compute the policy derivatives needed for that primal step.

Policy gradients and the likelihood-ratio identity. For task i, write the return of a trajectory as $G_i(\tau) = \sum_{t \geq 0} \gamma^t r_i(s_t, a_t)$. Only the action probabilities in the trajectory distribution depend on ϕ; the initial-state distribution and transition kernel do not. Hence

$$\begin{aligned} \nabla_\phi V_i^{\pi_\phi} &= \nabla_\phi \int p_i^{\pi_\phi}(\tau) G_i(\tau)\, d\tau \\ &= \int p_i^{\pi_\phi}(\tau) G_i(\tau) \nabla_\phi \log p_i^{\pi_\phi}(\tau)\, d\tau \\ &= \mathbb{E}_{\tau \sim p_i^{\pi_\phi}} \left[G_i(\tau) \sum_{t \geq 0} \nabla_\phi \log \pi_\phi(a_t \mid s_t) \right]. \end{aligned} \tag{13}$$

The second line uses $\nabla p = p \nabla \log p$, which is the likelihood-ratio identity. Equation (13) is the REINFORCE estimator: trajectories sampled from the current policy provide an unbiased Monte Carlo estimate of the gradient. In implementations, $G_i(\tau)$ may be replaced by a reward-to-go or an actor–critic advantage estimate to reduce variance; this does not alter the constrained objective. Since $V_i^{\pi_{\bar{G}}}$ is fixed during consolidation,

$$\nabla_\phi R_k(\phi) = \sum_{i=1}^{k-1} \omega_i \nabla_\phi V_i^{\pi_\phi}. \tag{14}$$

Derivative of the current-task constraint. Let $d_k(\phi) = V_k^{\pi_L} - V_k^{\pi_\phi}$. When $d_k(\phi) > 0$, the hinge in (6) is active and $F_k = d_k^2$. Because $V_k^{\pi_L}$ is fixed,

$$\nabla_\phi F_k = 2 d_k(\phi) \nabla_\phi d_k(\phi) = -2\big(V_k^{\pi_L} - V_k^{\pi_\phi}\big) \nabla_\phi V_k^{\pi_\phi}. \tag{15}$$

When $d_k(\phi) \leq 0$, the candidate already meets the local reference, $F_k = 0$, and its derivative is zero. Combining both cases gives

$$\nabla_\phi F_k(\pi_\phi) = -2 \left[V_k^{\pi_L} - V_k^{\pi_\phi} \right]_+ \nabla_\phi V_k^{\pi_\phi}. \tag{16}$$

Derivative of an old-task residual. For a rollout beginning at $s_0 = s$, define its sampled residual return

$$Z_{i,n} = \sum_{\ell=0}^{n-1} \gamma^\ell r_i(s_\ell, a_\ell) + \gamma^n \widehat{V}_i(s_n) - \widehat{V}_i(s_0) = \sum_{\ell=0}^{n-1} \gamma^\ell \delta_\ell. \tag{17}$$

Thus $\Delta_{i,n}(s;\phi) = \mathbb{E}[Z_{i,n} \mid s_0 = s]$. The score function at time j can affect only the suffix of the rollout. Its appropriate reward-to-go is therefore

$$Z^{\mathrm{rtg}}_{i,j} = \sum_{\ell=j}^{n-1} \gamma^{\ell-j} \delta_\ell. \tag{18}$$

This is *not* the usual advantage function: it is the discounted residual return remaining after action a_j. It plays the same role as a reward-to-go in REINFORCE. Applying the likelihood-ratio identity to (17) gives

$$\nabla_\phi \Delta_{i,n}(s;\phi) = \mathbb{E}\left[\sum_{j=0}^{n-1} \gamma^j Z^{\mathrm{rtg}}_{i,j} \nabla_\phi \log \pi_\phi(a_j \mid s_j) \,\middle|\, s_0 = s \right]. \tag{19}$$

The factor γ^j restores the discount applied to the suffix in the original residual. A state-dependent baseline may be subtracted from $Z^{\mathrm{rtg}}_{i,j}$ to reduce variance, provided that the baseline is independent of a_j.

For the maximum constraint, choose an active anchor $s_i^\star \in \arg\max_{s \in X_i} \{\tau_i - \Delta_{i,n}(s;\phi)\}$. Since τ_i is fixed,

$$\partial_\phi C_i(\phi) \ni -\nabla_\phi \Delta_{i,n}(s_i^\star;\phi). \tag{20}$$

Primal and dual updates. Substituting (14), (16), and (20) into $\partial_\phi \mathcal{L}$ gives the primal ascent direction

$$\partial_\phi \mathcal{L} \ni \sum_{i=1}^{k-1} \omega_i \nabla_\phi V_i^{\pi_\phi} + 2\mu \left[V_k^{\pi_L} - V_k^{\pi_\phi} \right]_+ \nabla_\phi V_k^{\pi_\phi} + \sum_{i=1}^{k-1} \lambda_i \nabla_\phi \Delta_{i,n}(s_i^\star;\phi). \tag{21}$$

The three terms respectively improve old-task return, restore current-task performance only when it falls below the local policy, and repair the most violated old-task anchor. With primal step size β, update $\phi \leftarrow \phi + \beta\, \widehat{\partial_\phi \mathcal{L}}$, where the hat denotes a Monte Carlo estimate.

For the dual variables, differentiate the Lagrangian directly:

$$\frac{\partial \mathcal{L}}{\partial \mu} = -(F_k - \epsilon), \qquad \frac{\partial \mathcal{L}}{\partial \lambda_i} = -C_i. \tag{22}$$

Because the dual problem is a minimization, gradient descent followed by projection onto the nonnegative orthant yields

$$\mu \leftarrow [\mu + \eta_\mu (F_k - \epsilon)]_+, \qquad \lambda_i \leftarrow [\lambda_i + \eta_\lambda C_i]_+. \tag{23}$$

For example, $F_k > \epsilon$ increases μ, while $C_i > 0$ increases λ_i; satisfied constraints allow their multipliers to decrease toward zero. In practice, all quantities are estimated from sampled trajectories, and a minibatch approximation to the maximum in (10) selects the most violated sampled anchor for each old task.

Stage-k procedure. Given π_G^-, M_k, and $\{(M_i, \widehat{V}_i)\}_{i<k}$, train a local actor–critic on M_k to obtain π_L and $\widehat{V}_k$. Generate temporary anchor sets using π_G^-, initialize $\phi \leftarrow \phi^-$ and $\mu, \lambda_i \leftarrow 0$, and repeat: estimate the full-return and residual terms, select each task's most violated anchor, update ϕ with (21), and update the multipliers with (23). Output $\pi_G^+ \leftarrow \pi_\phi$, retain $\widehat{V}_k$, and discard π_L and the temporary anchors.